SIVF: High-Performance Indexing for Streaming Vector Search with GPU-Resident Mutation

Dongfang Zhao
dzhao@uw.edu
High-Performance Data-Intelligence Computing (HPDIC) Lab
University of Washington, USA

Abstract

GPU-accelerated Approximate Nearest Neighbor (ANN) search has become the standard for large-scale vector similarity retrieval. However, existing GPU indices are predominantly designed for static, write-once-read-many workloads. In streaming scenarios such as real-time recommendation and security monitoring, where data must be frequently inserted and evicted to maintain a sliding window, current architectures suffer from a critical bottleneck: the lack of native support for in-place mutation on GPUs. Consequently, evicting data necessitates an expensive CPU-GPU roundtrip to reorganize the index layout, causing system latency to spike from milliseconds to seconds. In this paper, we propose SIVF (Streaming Inverted File), a GPU-native indexing architecture designed specifically for high-velocity streaming data. SIVF abandons the traditional contiguous memory layout in favor of a dynamic, slab-based allocation system combined with a validity bitmap, enabling efficient lock-free insertion and deletion directly in VRAM. To resolve the overhead of locating vectors for deletion, we introduce a GPU-resident address translation table (ATT) that provides $O(1)$ access to physical storage slots. We evaluate SIVF against state-of-the-art baselines on the SIFT1M and GIST1M datasets. Experimental results demonstrate that SIVF reduces deletion latency by up to $13,300\times$ (from 11.8 seconds to 0.89 ms on GIST1M) and improves ingestion throughput by $36\times$ to $105\times$ compared to the latest Faiss implementations. While sustaining competitive search recall, SIVF effectively eliminates the mutation bottleneck, paving the way for fully GPU-resident, real-time streaming vector analytics.

1 Introduction

Approximate Nearest Neighbor (ANN) search has become a foundational primitive for modern AI, powering everything from Retrieval-Augmented Generation (RAG) to real-time Large Language Model (LLM) serving. While GPU acceleration has successfully scaled static vector search to billions of items, the architecture of existing GPU indices remains fundamentally immutable, optimized almost exclusively for "write-once, read-many" workloads. This static design creates a critical bottleneck for emerging streaming applications, such as fraud detection and continuously updated memory systems, where data freshness is paramount and high-velocity deletion is as critical as ingestion. In this paper, we identify the *CPU-GPU Roundtrip* phenomenon as the root cause of this limitation, where simple deletion requests trigger a prohibitive full-index rebuild on the host. We introduce SIVF (Streaming Inverted File), a new GPU-native architecture that eliminates this bottleneck through slab-based memory management and atomic lazy eviction. By moving index maintenance entirely to the device, SIVF transforms vector search from a static retrieval task into a dynamic, real-time data management capability, achieving orders-of-magnitude improvements in mutation latency.

1.1 Observation: The Deletion Bottleneck

Real-world vector search applications, ranging from real-time recommendation engines to fraud detection systems, increasingly operate on streaming data where timeliness is critical. These systems necessitate a sliding window model where expired vectors must be invalidated promptly as new vectors arrive to maintain bounded memory usage. However, existing GPU-accelerated approximate nearest neighbor (ANN) indices are predominantly optimized for static, write-once-read-many workloads.

To quantify the gap between insertion and eviction performance, we conducted benchmarks using Faiss [18] on an NVIDIA RTX 6000 GPU with the SIFT1M dataset [20]. As illustrated in Figure 1, we observe a drastic performance asymmetry. While the GPU parallelism accelerates vector insertion, enabling reasonable throughput, the eviction process collapses under load. Specifically, deleting a batch of 10,000 vectors incurs a latency of over 1.6 seconds. In stark contrast, we will show that our proposed architecture performs the same operation in less than 0.9 milliseconds. This represents a slowdown of over three orders of magnitude for the baseline, confirming that the bottleneck is not merely an implementation detail but a fundamental structural deficiency in current index designs.

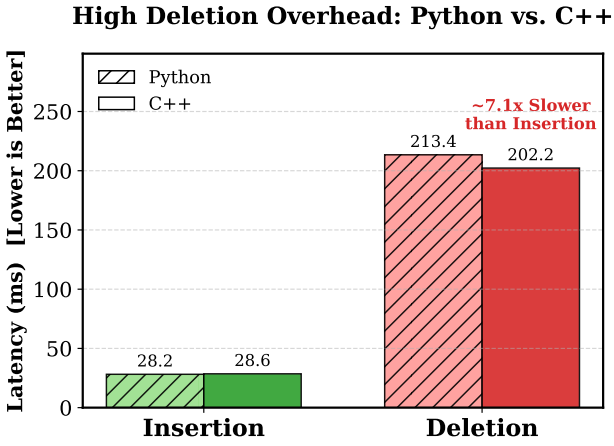

Figure 1: The asymmetry between insertion and deletion latency on GPU indices.

1.2 Root Cause: The Missing Operator

Our analysis of the Faiss source code [5] reveals that this performance asymmetry stems from a rigid class hierarchy that enforces a fallback to host-side processing. In the library's architecture, the data eviction interface is defined in the abstract base class `faiss::Index` via the `remove_ids` virtual method. While CPU-based implementations override this method to provide efficient in-memory deletion logic (e.g., `memmove`), the GPU counterparts inherit the base implementation which lacks native support.

Consequently, current GPU indices rely on a *CPU-GPU Roundtrip* pattern for eviction. That is, to delete even a single vector, the system must transfer the entire index state from VRAM to host memory, perform the compaction on the CPU, and re-upload the modified index to the device. This heavy I/O burden saturates the PCIe bus and prevents the system from utilizing GPU compute throughput, capping the maximum sustainable ingestion rate in streaming scenarios.

1.3 Technical Challenges

The absence of a GPU-native eviction operator is not an oversight but a consequence of the complexity involved in managing dynamic data structures within VRAM. Unlike CPU indices that utilize flexible dynamic arrays, GPU indices rely on pre-allocated, contiguous memory blocks to maximize memory coalescing and bandwidth utilization. Implementing in-place deletion on the GPU presents three orthogonal challenges.

First, maintaining contiguous arrays requires dynamic memory management. Frequent reallocation and compaction of inverted lists within VRAM lead to severe memory fragmentation and costly data movement. Second, concurrent modification requires complex synchronization. Managing fine-grained locks to prevent race conditions when thousands of concurrent GPU threads attempt to modify shared list structures introduces significant latency. Third, standard inverted file (IVF) indices lack an ID-to-Location mapping. Without a reverse index, locating a specific ID for deletion requires scanning all inverted lists, an operation with $O(N)$ complexity that negates the benefits of GPU acceleration. These constraints render naive in-place GPU eviction prohibitively inefficient.

1.4 Proposed Solution: The SIVF Architecture

To resolve these architectural limitations, we propose SIVF (Streaming Inverted File), a GPU indexing architecture designed to support high-throughput insertion and deletion without relying on host interaction. SIVF introduces a triad of mechanisms to enable efficient in-place mutability on GPUs.

We first address the memory management overhead by abandoning the contiguous memory layout in favor of a *slab allocation* system. GPU memory is pre-allocated as a pool of fixed-size blocks, and inverted lists are implemented as chained blocks. When a list grows, a new block is atomically linked from the free pool, virtually eliminating memory fragmentation and avoiding the cost of data compaction. To solve the synchronization challenge, we decouple logical deletion from physical removal using a bitmap-based lazy eviction strategy. Each memory block is associated with a validity bitmap; eviction is reduced to a single atomic bit-flip, allowing thousands of concurrent threads to operate without contention. Finally, to eliminate the $O(N)$ scan overhead, SIVF maintains a compact, GPU-resident address translation table (ATT). This table maps active vector IDs to their physical memory coordinates, enabling $O(1)$ location resolution during deletion.

1.5 Contributions

This paper makes three technical contributions to the field of high-performance vector search:

- **Quantifying the Deletion Bottleneck:** We identify the "CPU-GPU roundtrip bottleneck" in existing GPU ANN indices, demonstrating that the lack of native deletion support renders them unsuitable for high-velocity streaming data. Our benchmarks and analysis reveal that this architectural limitation causes latency spikes of orders of magnitude during index maintenance.
- **The SIVF Architecture:** We propose SIVF, a new GPU-native indexing architecture that synthesizes slab-based memory management, validity bitmaps, and on-device address translation. This design enables $O(1)$ time complexity for deletion and constant-time insertion overhead without costly CPU-GPU synchronization.
- **Orders-of-Magnitude Performance Gains:** We evaluate SIVF against state-of-the-art libraries on the multiple real-world datasets. Our results show that SIVF reduces deletion latency by up to $13,000\times$ (from 11.8s to 0.89ms) and improves ingestion throughput by over $36\times$, effectively closing the gap between static and streaming vector search performance on GPUs.

Paper Organization. The remainder of this paper is organized as follows. Section 2 reviews the existing landscape of ANN search, distinguishing between CPU-based dynamic methods and static GPU-accelerated indices. Section 3 details the architectural design of SIVF, elaborating on the slab-based memory allocator, the validity bitmap mechanism, and the GPU-resident address translation table. Section 4 presents our experimental methodology and a comprehensive evaluation of SIVF against state-of-the-art baselines. Finally, Section 5 concludes the paper and outlines directions for future research.

2 Related Work

Recent advancements in Approximate Nearest Neighbor (ANN) search have primarily focused on refining graph-based indexing structures, integrating attribute filtering for hybrid queries, and enhancing system scalability through quantization and hardware acceleration. In parallel, the high-performance computing (HPC) community has produced a complementary body of GPU systems research that studies how to restructure computation, memory layout, and runtime coordination to better match GPU execution and memory hierarchies. These GPU-oriented techniques provide reusable patterns, such as warp-aligned work partitioning, coalesced accesses, and low-overhead synchronization, that can inform the design of GPU-resident ANN systems.

2.1 Optimization of Graph-Based Indices

Graph-based indices remain the state-of-the-art for high-recall retrieval. To address performance bottlenecks in production, general frameworks like VSAG [45] optimize memory access patterns and parameter tuning, while SOAR [35] improves indexing structures. Navigational efficiency is a key optimization target; SHG [9] introduces a hierarchical graph with shortcuts to bypass redundant levels, and probabilistic routing methods [29] have been proposed to enhance traversal. Several works focus on distance metric optimizations: ADA-NNS [19] utilizes angular distance guidance to filter irrelevant neighbors, DADE [4] accelerates comparisons via data-aware distance estimation in lower dimensions, and HSCG [34] adapts the Monotonic Relative Neighbor Graph for cosine similarity using hemi-sphere centroids.

Efforts to improve graph construction and maintenance include LIGS [3], which employs locality-sensitive hashing (LSH) to simulate proximity graphs for faster updates, and CSPG [42], which crosses sparse proximity graphs. Addressing robustness, Hua et al. [11] propose dynamically detecting and fixing graph hardness for out-of-distribution queries. MIRAGE-ANNS [37] attempts to bridge the gap between incremental and refinement-based construction. Furthermore, theoretical frameworks like Subspace Collision [40] provide guarantees on result quality using clustering-based indexing.

2.2 Attribute-Filtered and Hybrid Search

The integration of structured constraints with vector search has led to a surge in Filtered ANN research. Li et al. [24] provide a comprehensive taxonomy and benchmark of these methods. A primary focus is handling dynamic updates and specific filter types such as ranges or windows. For range filtering, UNIFY [27] introduces a segmented inclusive graph to support pre-, post-, and hybrid filtering strategies. Dynamic environments are addressed by DIGRA [17], which uses a multi-way tree structure for dynamic graph indexing, and RangePQ [44], which offers a linear-space indexing scheme for range-filtered updates. Peng et al. [32] propose dynamic segment graphs to handle mixed streams of data and queries.

Regarding specific constraints, Wang et al. [38] present a robust framework for general attribute constraints. Engels et al. [6] focus on window filters, while WoW [39] develops a window-graph based index to handle window-to-window incremental construction.

2.3 Quantization and Scalability

To manage high-dimensional data at scale, quantization and system-level optimizations are critical. RaBitQ [8] and its extended version [7] provide theoretical error bounds for bitwise quantization with flexible compression rates. SymphonyQG [10] integrates quantization codes directly with graph structures to reduce memory access overhead, while LoRANN [16] utilizes low-rank matrix factorization for compression.

In terms of system architecture, Tagore [26] leverages GPUs for accelerating graph index construction. For distributed environments, HARMONY [41] employs a multi-granularity partition strategy to balance load. WebANNS [28] enables efficient search within web browsers via WebAssembly. From a theoretical perspective, Indyk and Xu [15] analyze the worst-case performance limits of popular implementations. Finally, DARTH [1] introduces declarative recall targets through adaptive early termination to balance performance and result quality.

2.4 GPU Systems and Optimizations

Recent HPC work highlights how careful kernel and data-layout design can unlock GPU performance across data-intensive workloads. FZ-GPU demonstrates a fully parallel compression pipeline with both warp-aware bitwise operations and shared-memory efficiency [43]. For sparse linear algebra, SpMV designs reduce preprocessing overhead while improving load balance and memory access locality [2], and GPU-aware preconditioning further emphasizes locality and coalescence as first-order concerns on modern GPU architectures [22]. Beyond compute kernels, GPU-enabled asynchronous checkpoint caching and prefetching treat GPU memory as a first-class tier to improve end-to-end throughput for I/O-heavy workflows [30]. Format choices and composition are also critical for irregular workloads, as shown by automatic sparse format composition for SpMM that avoids expensive autotuning while delivering strong performance [33].

Another line of work focuses on reliability, debugging, and cluster-scale GPU management, which together shape how GPU-centric systems are built and operated. GPU-FPX provides low-overhead floating-point exception tracking for NVIDIA GPUs [25], FPBOXer improves scalable input generation for triggering floating-point exceptions in GPU programs [36], and FloatGuard extends whole-program exception detection to AMD GPUs and highlights cross-vendor portability concerns [31]. At the application and cluster level, SIMCoV-GPU studies multi-node, multi-GPU acceleration for irregular agent-based simulations [23], FASOP automates fast search for near-optimal transformer parallelization on heterogeneous GPU clusters [14], and serverless platforms motivate GPU sharing and scheduling mechanisms such as ESG and FluidFaaS [12, 13]. These works reinforce the importance of aligning work granularity with warps, minimizing synchronization overhead, and designing memory layouts that preserve locality, which are relevant to GPU-resident indexing under dynamic updates.

3 Design and Implementation of SIVF

In this section, we detail the architecture of SIVF, designed to enable high-concurrency mutability on GPUs. We begin by introducing the Slab-based Dynamic Memory Allocator (Section 3.1), which replaces rigid contiguous arrays with a flexible pool of fixed-size blocks to mitigate fragmentation. Building on this storage layer, we describe the Lock-free Parallel Ingestion protocol (Section 3.2) that allows conflict-free streaming updates via atomic slot reservation. To ensure retrieval efficiency despite the non-contiguous memory layout, we present a specialized High-Performance Search Kernel (Section 3.3) utilizing warp-level collaboration. Finally, we detail the Bitmap-based Lazy Eviction mechanism (Section 3.4), which achieves $O(1)$ deletion complexity through a GPU-resident address translation table.

3.1 Slab-based Memory Management

Standard GPU-based IVF indices rely on contiguous memory arrays to store inverted lists. This monolithic design exhibits severe

limitations in streaming scenarios. First, resizing an inverted list requires allocating a larger memory region and copying existing data, introducing synchronization stalls and high latency. Second, frequent insertions and deletions of variable-length lists exacerbate memory fragmentation and reduce allocator efficiency on GPUs.

To address these challenges, we propose a Slab-based Dynamic Memory Allocator (SDMA). Instead of managing variable-length arrays, SDMA pre-allocates a contiguous GPU memory region and manages it as a pool of fixed-size blocks, referred to as *slabs*. Inverted lists are therefore represented as linked chains of slabs. This design transforms dynamic list growth into constant-time block allocation while enabling efficient in-place mutation through lightweight metadata updates.

3.1.1 Slab Layout and Metadata. A *slab* is defined as the fundamental unit of storage. Each slab has a fixed capacity C, and we set $C = 32$ to align with the NVIDIA warp size, thereby enabling coalesced memory access when threads within a warp operate on the same slab. A Slab S_i consists of two regions:

- *Data Payload*: Stores vectors contiguously. For a vector space of dimension D, the payload size is
$$L_p = C \times D \times \text{sizeof(float)}.$$
- *Metadata Header*: Stores control information required for traversal and mutation. The metadata of slab S_i is represented as
$$M_i = \langle n_{next}, b_{valid}, c_{valid} \rangle, \tag{1}$$
where n_{next} denotes the identifier of the next slab in the logical list, b_{valid} is a 32-bit validity bitmap indicating occupied slots, and c_{valid} records the number of valid vectors stored in the slab.

Logical deletion is implemented by atomically clearing the corresponding bit in b_{valid}, yielding constant-time removal without triggering memory compaction or data movement to CPUs.

3.1.2 Conflict-Free Slab Allocation. General-purpose device-side memory allocators incur substantial synchronization and fragmentation overhead when invoked from GPU kernels. SDMA instead manages slabs from a pre-allocated pool. On initialization, a device kernel constructs a free-list array and sets a global stack pointer P_{top} to the pool size. Allocation obtains a fresh slab identifier by atomically decrementing P_{top}:

$$s_{id} = \texttt{atomicSub}(P_{top}, 1) - 1. \tag{2}$$

In lock-free allocators, reusing memory identifiers may lead to the "ABA" problem, where a shared variable changes from value A to B and back to A between two observations, causing concurrent threads to falsely assume that the system state remains unchanged. In GPU memory management, this may occur if a slab identifier is reclaimed and reassigned while another thread still holds a stale reference. SDMA avoids this issue by ensuring that slab identifiers are never reused while concurrent kernels may still observe them. Deletion only invalidates logical occupancy via the bitmap and does not immediately recycle slab identifiers back to the pool. This design keeps the meaning of an allocated slab identifier stable during execution and avoids additional versioning mechanisms.

3.1.3 Virtual Addressing via Address Table. To decouple logical vector identifiers from physical storage locations, SDMA introduces a global Address Table that implements a translation function

$$\mathcal{T}(v_{id}) \rightarrow \langle s_{id}, o_{slot} \rangle, \tag{3}$$

where s_{id} denotes the slab identifier and o_{slot} denotes the slot offset within that slab, with $0 \leq o_{slot} < C$.

The address table is initialized with an invalid sentinel value for all entries. Upon insertion, the table is updated to record the physical location of each vector. During deletion, the system queries $\mathcal{T}(v_{id})$ to directly locate the corresponding slab and slot, enabling constant-time invalidation without scanning inverted lists.

3.1.4 System Implementation. We implemented SDMA in CUDA C++ within the Faiss framework using a dual-view architecture. On the host side, GPU memory resources are managed using RAII-compliant containers encapsulated in a `SlabManager` class, including slab metadata, slab data, address table, and allocator state. These resources are projected into a lightweight `SlabManagerDevice` structure containing raw pointers and scalar parameters passed by value to device kernels.

Index integration is implemented in `GpuIndexSIVF`. During initialization, the index selects a safe slab pool size and derives a consistent maximum vector capacity to prevent device memory overflow when slabs are appended. The index maintains an array of per-list head pointers (`list_heads`) initialized to an invalid value. Insertions first perform coarse quantization using the existing Faiss GPU quantizer to obtain list assignments, and then invoke a GPU append kernel that writes vectors into the slab chains. Deletions use the address table to locate and invalidate target slots in place.

Algorithm 1 summarizes the allocator and deletion logic. Insertion resolves the target inverted list, attempts to reserve a free slot in the current head slab, and allocates and links a new slab if needed. The address table is updated after the physical location is finalized. Deletion performs a direct address table lookup followed by an atomic bit clear, leaving slab storage intact.

3.2 Lock-free Parallel Ingestion

Many GPU-based ANN indices are optimized for static or bulk ingestion and commonly rely on sorting, segmented scans, or bulk-copying to build inverted lists. While these approaches can achieve high throughput, they often incur high end-to-end latency for streaming updates. SIVF instead implements a fully parallel insertion kernel in which each CUDA thread ingests one vector and performs lock-free appends to inverted lists. Concurrency is handled using atomic slot reservation in the current head slab and speculative slab expansion under contention. Correctness under CUDA's relaxed memory consistency is ensured by device-wide memory fences and a publish protocol based on a validity bitmap.

At a high level, each SIVF list is represented as a singly linked chain of fixed-capacity slabs. Each slab stores up to $C = 32$ vectors and maintains a counter `valid_count`, a 32-bit `validity_bitmap`, and a link field `next_slab_idx`. Insertion follows a two-stage protocol. First, a thread attempts to reserve a slot in the current head slab. If the head slab is absent or full, the thread allocates a new slab from the global pool and attempts to publish it as the new head using compare-and-swap (CAS). In both cases, the inserted vector

Algorithm 1 Core Operations of SDMA

Global State: slab pool metadata and data, per-list head pointers $H[\cdot]$, stack pointer P_{top}, address table $\mathcal{T}$
Input: vector identifier v_{id}, vector x, list assignment ℓ
procedure INSERT(v_{id}, x, ℓ)
    $s \leftarrow H[\ell]$
    while s is invalid **or** no free slot exists in slab s **do**
        $s_{new} \leftarrow \texttt{atomicSub}(P_{top}, 1) - 1$
        Initialize metadata s_{new} with $b_{valid} = 0$, $n_{next} = s$
        Update $H[\ell]$ to s_{new} if unchanged, otherwise retry
        $s \leftarrow H[\ell]$
    end while
    Reserve a free slot o in slab s via atomic update of c_{valid}
    Write x to slab data$[s][o]$
    Atomically set bit o in b_{valid}
    $\mathcal{T}(v_{id}) \leftarrow \langle s, o \rangle$
end procedure
procedure DELETE(v_{id})
    $\langle s, o \rangle \leftarrow \mathcal{T}(v_{id})$
    if $\langle s, o \rangle$ is valid **then**
        Atomically clear bit o in b_{valid} of slab s
        Mark $\mathcal{T}(v_{id})$ as invalid
    end if
end procedure

becomes visible to concurrent readers only when the corresponding validity bit is set after a memory fence.

3.2.1 Atomic Slot Reservation. Ingestion begins with coarse quantization, assigning each vector to a list ℓ. A thread reads the current head slab index $h = H[\ell]$. If $h \neq -1$, the thread attempts to reserve a writing slot by conditionally incrementing the head slab counter. Let c be the value of `valid_count`. If $c < C$, the thread executes:

$$\text{success} = \texttt{atomicCAS}(\&\text{slab}[h].\text{valid\_count}, c, c+1) == c. \quad (4)$$

When the above check succeeds, the reserved slot index is $o = c$. This conditional CAS-based increment ensures that exactly one thread claims each slot index and avoids counter overflow under contention. Threads that fail the CAS retry by re-reading the head and counter.

After reserving a slot, the thread writes the vector payload into the slab data region, writes the user identifier into a parallel identifier buffer indexed by `(slab, slot)`, and updates the address table entry for the user identifier to a compact coordinate encoding $\langle s_{id}, o_{slot} \rangle$. Only after these writes are completed does the thread execute `__threadfence()` and set the corresponding bit in `validity_bitmap` via `atomicOr`. The validity bit is the publication signal for readers, so this ordering ensures that a concurrent search that observes an enabled bit never reads uninitialized payload or a missing address translation.

3.2.2 Speculative Slab Expansion. If the head slab is absent ($h = -1$) or full, the thread expands the list by allocating a new slab from the global pool. Allocation decrements a global stack pointer `free_list_top` and retrieves a slab index from `free_list`. If the pool is exhausted, the thread restores the pointer and returns.

After allocation, the thread initializes the new slab metadata by setting `valid_count` to 1, clearing `validity_bitmap` to 0, and setting `next_slab_idx` to the previous head h. A `__threadfence()` is executed before attempting to publish the new slab as the list head using CAS:

$$\text{success} = \texttt{atomicCAS}(\&H[\ell], h, s_{new}) == h. \quad (5)$$

Equation 5 defines the linearization point of list expansion. Only one contending thread succeeds in updating the head pointer, and its newly allocated slab becomes immediately visible to concurrent writers and readers that traverse from the head.

If the head publication CAS fails due to contention, the allocated slab is not returned to the free list. Instead, the thread retries the insertion loop. This leak-on-failure policy avoids additional synchronization on allocator state under contention and prevents hazards that may arise from reclaiming slab identifiers while other threads may still observe stale references. In practice, the overhead is controlled by provisioning sufficient slab pool capacity so that occasional contention-induced leakage does not affect ingestion.

3.2.3 Kernel-Level Memory Ordering and Publication. SIVF enforces safe publication under CUDA's weak memory consistency using `__threadfence()` at two critical points. First, before a thread sets a slot's validity bit, it fences to ensure that payload writes, identifier buffer writes, and address table updates become globally visible. Second, before a thread publishes a newly initialized slab as the list head, it fences to ensure that slab metadata is visible before any concurrent traversal observes the head update. These fences prevent readers from observing a published structural pointer or validity bit that refers to partially initialized state.

3.2.4 Micro-architectural Considerations.

Register-efficient addressing. SIVF represents slab links using 32-bit slab indices within a pre-allocated region rather than 64-bit device pointers. This reduces register pressure and simplifies address computation in the critical path. The address table stores a compact 64-bit coordinate encoding the slab index and slot offset, enabling constant-time deletion without list scans.

Contention behavior and forward progress. Insertion experiences high contention when many vectors map to the same centroid. Most insertions complete through slot reservation when the head slab has available capacity. When the head becomes full, contention shifts to head publication, after which subsequent threads quickly observe the new head and resume slot reservation. The kernel bounds retries with a fixed attempt limit.

Intra-slab coalescing. Vectors within each slab are stored contiguously. When multiple threads append to adjacent slots, payload writes to slab data and identifier buffer writes are naturally coalesced, preserving memory bandwidth under streaming ingestion.

Algorithm 2 summarizes the per-thread ingestion protocol implemented by the insertion kernel. The protocol first attempts slot reservation on the current head slab using CAS on `valid_count`. If successful, it writes payload, identifier, and address table entry, then publishes the slot by setting the validity bit after a device-wide fence. If the head is absent or full, it allocates and initializes a new slab and attempts to publish it as the new head using CAS

Algorithm 2 Lock-free ingestion protocol in SIVF

Input: vector x, identifier v_{id}, list assignment ℓ
Global: head array $H[\cdot]$, slab metadata, slab data, free list, stack pointer P_{top}, Address Table $\mathcal{T}$
$attempts \leftarrow 0$
while $attempts < 1000$ **do**
  $attempts \leftarrow attempts + 1$
  $h \leftarrow H[\ell]$
  if $h \neq -1$ **then**
    $c \leftarrow$ slab_meta[h].valid_count
    if $c < C$ **then**
      if `atomicCAS`(&slab[h].valid_cnt) == c **then**
        Write payload x to slab data[h][c]
        Write identifier to slab id buffer[h][c]
        $\mathcal{T}(v_{id}) \leftarrow \langle h, c \rangle$
        `__threadfence()`
        `atomicOr`(&slab_meta[h].validity_bitmap)
        return
      end if
    end if
  end if
  $t \leftarrow$ `atomicSub`(P_{top}, 1)
  if $t \leq 0$ **then**
    `atomicAdd`(P_{top}, 1)
    return
  end if
  $s_{new} \leftarrow$ free_list[$t - 1$]
  Set slab_meta[s_{new}].valid_count $\leftarrow 1$
  Set slab_meta[s_{new}].validity_bitmap $\leftarrow 0$
  Set slab_meta[s_{new}].next $\leftarrow h$
  `__threadfence()`
  if `atomicCAS`($&H[\ell], h, s_{new}$) == h **then**
    Write payload x to slab data[s_{new}][0]
    Write identifier to slab id buffer[s_{new}][0]
    $\mathcal{T}(v_{id}) \leftarrow \langle s_{new}, 0 \rangle$
    `__threadfence()`
    `atomicOr(&slab_meta[s_new].validity_bitmap)`
    return
  end if
end while

on the head pointer, again with a fence before publication. If head publication fails, it retries without reclaiming the allocated slab.

3.3 High-Performance Search Kernel

SIVF replaces contiguous inverted lists with linked slabs, which introduces pointer chasing during traversal. To retain high throughput on GPUs, we design a warp-cooperative search kernel that maps the slab capacity to the warp width. The kernel is optimized for the execution model in which one warp collaboratively processes one query and evaluates one slab per traversal step.

3.3.1 Warp-Cooperative Slab Traversal. SIVF launches the search kernel with one thread block per query and 32 threads per block. This design assigns each query to a single warp. For a query vector $\mathbf{q} \in \mathbb{R}^D$, the warp first stages the query into shared memory to reduce redundant global loads:

$$\mathbf{q} \rightarrow \texttt{shared\_query}.$$

The kernel then probes *nprobe* coarse lists. For each probed list, the warp traverses the slab chain starting from the list head. At each slab S, thread t_j is responsible for slot j in that slab. The thread consults the slab validity bitmap and only evaluates the distance if the corresponding bit is set:

$$\text{valid}(S, j) \Leftrightarrow ((b_{valid} \gg j)\ \&\ 1) == 1. \tag{6}$$

If valid, the thread loads the vector stored at (S, j) and computes the squared L_2 distance:

$$d(\mathbf{q}, \mathbf{x}_{S,j}) = \sum_{k=1}^{D} (q_k - x_{S,j,k})^2. \tag{7}$$

The slab pointer is advanced using the metadata `next_slab_idx`. The kernel also includes safety checks that bound the traversal length and break on degenerate self-loops, preventing potential non-terminating traversal under unexpected corruption.

3.3.2 Top-k Maintenance and Warp Aggregation. Maintaining top-k results without global contention is achieved using a two-stage aggregation scheme. Each thread maintains a small top-k list in registers. Upon computing a candidate distance, the thread inserts it into its local top-k list using an in-register insertion routine. This avoids shared data structures and eliminates inter-thread contention during candidate generation.

After all probed lists are traversed, the warp aggregates results. Each thread writes its local top-k list to shared memory. A single thread then performs a deterministic merge across the 32 per-thread lists and outputs the final top-k for the query. This design trades a small amount of serial work for a contention-free aggregation path and avoids additional warp-level primitives.

3.3.3 Correctness Under Concurrent Ingestion. SIVF supports concurrent ingestion by using a publish protocol based on the validity bitmap. The ingestion kernel writes the payload, updates the identifier buffer and address table entry, and then executes `__threadfence()` before setting the corresponding validity bit via an atomic operation. The search kernel consults the validity bit as the sole signal that a slot is present. Therefore, if search observes a valid bit, the payload and identifier for that slot have been made globally visible prior to publication. This ensures that search does not read partially initialized vectors from concurrent insertions.

Algorithm 3 describes the warp-cooperative search procedure of SIVF. We launch one CUDA block per query with 32 threads, so each query is processed by a single warp. At the beginning of the kernel, the warp cooperatively stages the query vector into shared memory to avoid redundant global loads during list traversal. For each query, we probe *nprobe* coarse lists returned by the quantizer. For a probed list, the warp traverses its slab chain starting from the list head pointer. At each slab, thread t_j corresponds to slot j and consults the slab validity bitmap. If the j-th bit is set, the thread loads the vector payload from the slab data region, computes the squared L_2 distance to the shared query, retrieves the corresponding user identifier from the slab identifier buffer, and updates its local top-k candidates. Traversal proceeds by following the `next_slab_idx` pointer recorded in the slab metadata. To ensure termination under

Algorithm 3 Warp-cooperative search in SIVF

Input: queries Q, probed list ids *coarse_ids*, head array $H[\cdot]$
Input: slab metadata and data, buffer *slab_ids*, k, *nprobe*
Output: top-k distances and labels for each query
for each query index q in parallel **do**
  Launch one warp for query q
  Copy $Q[q]$ into shared memory
  Initialize per-thread local top-k lists in registers
  for $p = 0$ to $nprobe - 1$ **do**
    $\ell \leftarrow coarse\_ids[q, p]$
    if ℓ is invalid **then**
      continue
    end if
    $s \leftarrow H[\ell]$
    while s is valid and traversal bound not exceeded **do**
      $md \leftarrow slab\_meta[s]$
      if $md.next = s$ **then**
        break
      end if
      if $j < 32$ and $md.validity\_bitmap[j]$ is set **then**
        Load vector $\mathbf{x}_{s,j}$ from slab data
        Compute $d(\mathbf{q}, \mathbf{x}_{s,j})$
        $id \leftarrow slab\_ids[s, j]$
        Insert (d, id) into local top-k
      end if
      $s \leftarrow md.next$
    end while
  end for
  Write local top-k lists to shared memory
  One thread merges 32 lists and writes final top-k outputs
end for

unexpected corruption, we additionally bound the traversal length and break on self-loops.

3.4 Bitmap-based Lazy Eviction

Traditional vector deletion in IVF indices often requires locating the target element inside an inverted list and physically compacting memory to close the gap, which can incur a cost proportional to the list length and trigger substantial global memory traffic. This overhead fundamentally limits deletion throughput in streaming workloads. SIVF adopts bitmap-based lazy eviction, which decouples logical validity from physical residency. Deletion is reduced to a constant-time metadata update by combining a global Address Translation Table (ATT) with atomic bitwise invalidation of per-slab validity bitmaps.

3.4.1 Address Translation Table (ATT). SIVF maintains a global table $\mathcal{T}$ that maps a user identifier u to its physical coordinate in the slab pool. The table is implemented as a flat array of 64-bit entries. For an identifier u, $\mathcal{T}[u]$ encodes the slab index and slot offset as:

$$\mathcal{T}[u] = (\mathrm{idx}_{slab} \ll 32) \mid \mathrm{idx}_{slot}, \tag{8}$$

where $\mathrm{idx}_{slot} \in [0, 31]$. The table is initialized with an invalid sentinel value, denoted as INVALID, for all entries. This design enables direct random access to the target slot without traversing inverted lists.

3.4.2 Atomic Bitwise Invalidation. Given $\mathcal{T}[u]$, the deletion kernel resolves $(\mathrm{idx}_{slab}, \mathrm{idx}_{slot})$ and invalidates the corresponding slot via the slab validity bitmap. Let $\mathcal{B}_i$ denote the 32-bit bitmap of slab S_i. The deletion operation clears the target bit using an atomic read-modify-write:

$$\mathcal{B}_i \leftarrow \texttt{atomicAnd}(\mathcal{B}_i, \sim (1 \ll \mathrm{idx}_{slot})). \tag{9}$$

The vector payload is not moved or overwritten. Search kernels consult the bitmap when scanning a slab and ignore slots whose bits are cleared, so deletion immediately removes the vector from the search space without compaction.

3.4.3 Idempotence and Consistency. Deletion requests may include duplicates or may race with other deletions. SIVF treats deletion as an idempotent operation. After the atomic bitmap update, the kernel checks whether the bit was previously set by inspecting the pre-update value returned by `atomicAnd`. Only if the bit transitioned from 1 to 0 does the kernel decrement the slab counter `valid_count` and increment a global deletion counter. Finally, the kernel marks $\mathcal{T}[u] \leftarrow$ INVALID to prevent future deletion requests from dereferencing a stale coordinate.

This protocol provides a clear correctness contract with concurrent search and insertion. Search considers a slot present only if its validity bit is set. Insertion sets the validity bit only after payload and identifier writes are completed and a device-wide memory fence is executed. Deletion clears the validity bit via an atomic operation, so a deleted slot is excluded from search as soon as the bit is cleared. The ATT invalidation is a secondary safety step that prevents repeated deletions from accessing recycled or stale coordinates.

Deletion performs a constant number of random memory accesses: one read of $\mathcal{T}[u]$, one atomic update of a 32-bit bitmap, and one write of the 64-bit ATT entry to the invalid sentinel. The data payload remains untouched, avoiding bandwidth-heavy compaction. In addition, the implementation optionally updates a 32-bit counter in the slab metadata and a global 32-bit deletion counter for bookkeeping. Overall, the dominant deletion work is independent of the inverted list length, explaining the large speedups observed in our evaluation.

Algorithm 4 presents the bitmap-based lazy eviction procedure implemented by the SIVF deletion kernel. Each GPU thread processes one identifier u in the deletion batch. The thread first performs a constant-time lookup in the ATT to obtain the encoded coordinate $\mathcal{T}[u]$. If the entry equals the invalid sentinel, the identifier is already absent and the thread terminates without side effects. Otherwise, the thread decodes the slab index and slot offset from the 64-bit coordinate, locates the corresponding slab metadata, and clears the target slot by applying an atomic bitwise operation to the slab validity bitmap. The atomic operation returns the previous bitmap value, which allows the kernel to determine whether the bit was previously set. Only when the bit transitions from 1 to 0 does the kernel treat the deletion as successful and perform two bookkeeping updates: it atomically decrements the slab `valid_count` and atomically increments a global `deleted_count`. Finally, the

Algorithm 4 Bitmap-based lazy eviction in SIVF

```
Input: deletion identifiers U = {u_1, ..., u_m}
Global: Address Table T, slab metadata with bitmap B and counter valid_count
for each u in U in parallel do
    coord ← T[u]
    if coord = INVALID then
        continue
    end if
    idx_slab ← coord ≫ 32
    idx_slot ← coord & (2^32 − 1)
    mask ←  ~ (1 ≪ idx_slot)
    old ← atomicAnd(B_{idx_slab}, mask)
    if ((old ≫ idx_slot) & 1) = 1 then
        atomicSub(&valid_count, 1)
        atomicAdd(&deleted_count, 1)
        T[u] ← INVALID
    end if
end for
```

kernel writes the invalid sentinel back to $\mathcal{T}[u]$ to prevent future deletion requests from dereferencing a stale coordinate.

4 Evaluation

We implement SIVF by extending the GPU components of the widely-adopted Faiss library [5], integrating our proposed slab-based memory management and lock-free update mechanisms into its core indexing architecture. Our evaluation benchmarks SIVF directly against the native Faiss GPU implementations, specifically targeting ingestion throughput and deletion latency under high-concurrency streaming workloads. We have open-sourced our complete implementation and evaluation scripts in the *ElasticIVF* repository on GitHub[1].

Implementing SIVF required substantial modifications to the Faiss GPU backend and associated experimental infrastructure. In total, the implementation involves approximately 9,000 lines of code changes across more than 45 source files. Core system changes are concentrated in the GPU index implementation, incorporating new GPU-native insertion, deletion, and search operators (e.g., `GpuIndexSIVF`, `SIVFAppend`, `SIVFSearch`), alongside a custom slab-based memory manager (`SlabManager`) and lock-free metadata structures. Furthermore, we developed a comprehensive benchmarking suite that extends beyond standard metric testing to include end-to-end streaming simulations, memory scalability analysis, and automated visualization scripts, ensuring a rigorous and reproducible evaluation.

4.1 Experimental Setup

All experiments were conducted on a bare-metal node from the Chameleon Cloud testbed [21] (CHI@UC site). The platform is equipped with dual-socket Intel Xeon Gold 6126 CPUs (Skylake microarchitecture), providing a total of 24 physical cores (48 threads with Hyper-Threading) clocked at 2.60 GHz. The host memory consists of 192 GiB of RAM. For acceleration, the node features an NVIDIA Quadro RTX 6000 GPU with 24 GB of GDDR6 memory. The system runs on a Ubuntu 24.04 environment with the CUDA toolkit installed.

To ensure statistical reliability, each reported data point represents the average of at least three independent executions. Error bars are omitted in figures where the standard deviation is negligible to maintain visual clarity. Unless explicitly stated (as in Section 4.5), our experiments utilize synthetic datasets consisting of random vectors generated from a uniform distribution. We employ this configuration as a neutral baseline to isolate system-level overheads (e.g., memory allocation, PCIe transfer) from data distribution effects. Note that because uniform data lacks the clustering structure inherent in real-world applications, the performance advantages of SIVF in microbenchmarks are primarily architectural and appear more conservative than those observed on real-world datasets.

4.2 Microbenchmark: Ingestion

We evaluate the ingestion throughput of SIVF against the baseline method, the standard Faiss GPU IVFFlat implementation. We measure the insertion throughput (vectors per second, vec/s) while varying two key parameters: the database size (N_B) from 1M to 4M vectors, and the number of clusters (n_{list}) from 1024 to 16384. The results are summarized in Figure 2. SIVF demonstrates consistent performance advantages across all configurations, achieving up to a 2.88× speedup over the baseline.

Scalability and Stability (Fig. 2a). As the database size grows from 1 million to 4 million vectors (with $n_{list} = 4096$), SIVF maintains a robust throughput of approximately 4.2 million vec/s. This stability verifies the efficiency of our adaptive SlabManager: despite the increasing memory footprint, the lock-free append mechanism ensures that insertion cost remains $O(1)$ and effectively decoupled from the current index occupancy. In contrast, the baseline operates at a significantly lower tier (≈1.7M vec/s) due to contiguous memory maintenance overheads.

Impact of Clustering Granularity (Fig. 2b). We observe that ingestion throughput is sensitive to the number of clusters n_{list}:

- Peak Performance at Low n_{list}: SIVF achieves its highest throughput (≈ 6.1M vec/s) at $n_{list} = 1024$ with 2M vectors. In this setup, the memory write patterns are more localized, reducing cache misses and maximizing the efficiency of our append kernels. SIVF is 2.88× faster than the baseline (≈2.1M vec/s).
- Degradation at High n_{list}: As n_{list} increases to 16384, the insertion pattern becomes highly scattered across thousands of memory slabs. Consequently, throughput for both systems decreases. However, the baseline suffers more severely from this fragmentation. SIVF maintains a throughput of ≈1.3M–1.6M vec/s, achieving a 2.0× to 2.4× speedup even in this challenging high-granularity scenario. This demonstrates the resilience of the linked-slab architecture compared to the baseline's array resizing overhead.

Overall Speedup (Fig. 2c). The heatmap confirms that SIVF outperforms the baseline in every tested configuration. The speedup is robust, typically ranging from 2.4× to 2.9× for most configurations. Even under the highest load (4M vectors with $n_{list} = 1024$), where

[1] https://github.com/hpdic/ElasticIVF

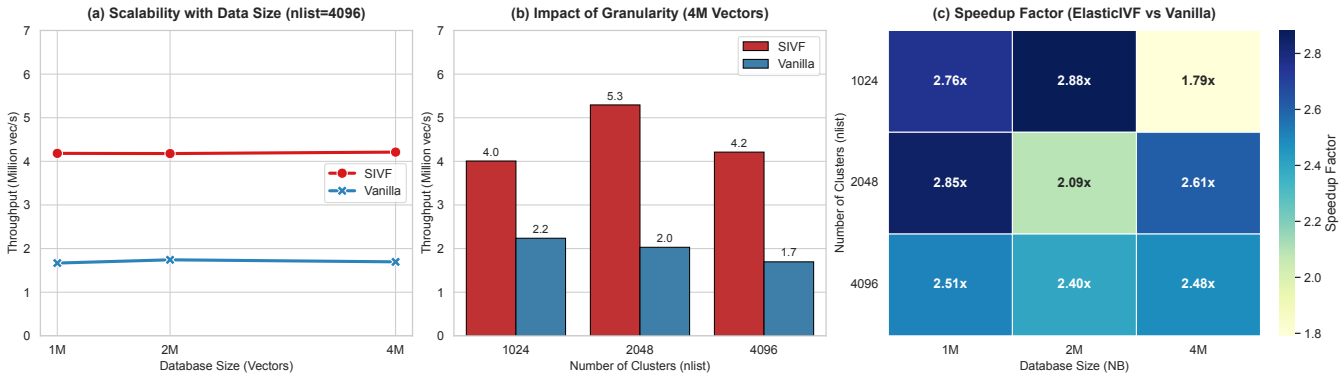

Figure 2: Performance evaluation of vector ingestion.

contention for slab heads is maximized, SIVF maintains a 1.79× advantage. This performance gap highlights the efficiency of our non-blocking, slab-based ingestion pipeline, making it well-suited for high-throughput streaming scenarios.

4.3 Microbenchmark: Search

In this section, we evaluate the query performance of SIVF against the Vanilla Faiss GPU implementation. Theoretically, SIVF is expected to exhibit a slight degradation in search throughput compared to the baseline, as its linked-slab architecture sacrifices memory contiguity (and thus perfect memory coalescing) to enable highly efficient, fine-grained updates. Our objective is to verify that this structural overhead remains within an acceptable range that does not compromise overall system utility. As summarized in Figure 3, the results confirm that SIVF maintains competitive search efficiency, reaching up to 91% of the baseline performance in optimized configurations, demonstrating that the cost of supporting instant mutability is reasonable.

Search Scalability (Fig. 3a). As the database size increases from 100,000 to 500,000 vectors (with $n_{list} = 4096$), SIVF throughput transitions from approximately 383k QPS to 118k QPS. The performance curve closely mirrors the baseline, confirming that our slab traversal logic maintains predictable scaling characteristics. Even at the 500k scale, SIVF provides sufficient throughput for high-concurrency production environments.

Impact of Granularity (Fig. 3b). At the 500k scale, increasing n_{list} from 1024 to 4096 significantly enhances SIVF performance from 70k to 118k QPS. This result is consistent with the principle that higher granularity reduces the average number of vectors per inverted list, thereby decreasing the total distance computations per probe. The throughput remains robust across all settings.

Query Efficiency Analysis (Fig. 3c). The efficiency ratio, defined as $QPS_{SIVF}/QPS_{Vanilla}$, reveals the effectiveness of our warp-level optimization:

- Near-Native Efficiency: At 100k vectors and $n_{list} = 16384$, SIVF achieves an efficiency ratio of 0.91× (295k vs. 322k QPS), nearly matching the performance of the contiguous-array-based baseline. This demonstrates that our warp-parallel search kernel effectively hides the latency associated with linked-slab traversal in sparse, high-granularity scenarios where lists are short.
- Impact of List Length: As the database size increases and inverted lists grow longer (e.g., at 500k vectors), the ratio stabilizes between 0.40× and 0.52×. This expected overhead is primarily due to increased pointer chasing between slabs, which is the physical trade-off required to enable the index's dynamic mutability.

4.4 Microbenchmark: Deletion

We evaluate the efficiency of the vector deletion mechanism in SIVF by comparing it against the standard Faiss GPU IVF baseline. The experiment measures latency and throughput when removing a batch of 10,000 vectors from a populated index of one million 128-dimensional vectors.

Figure 4 illustrates the performance comparison between the two methods. The results indicate that the baseline Faiss implementation incurs a substantial deletion latency of approximately 202.20 ms. In stark contrast, SIVF completes the same batch deletion operation in an average of 0.68 ms. This represents a massive speedup of 298.5×. Correspondingly, deletion throughput surges from 4.9×10^4 vec/s in the baseline to nearly 1.47×10^7 vec/s in SIVF.

4.5 Real-world Datasets

To evaluate SIVF under realistic conditions, we utilized two standard large-scale benchmarks: SIFT1M [20] (128 dimensions) and GIST1M [20] (960 dimensions). SIFT1M is a standard computer vision dataset using Euclidean distance; GIST1M is a high-dimensional dataset representing global image features, which is particularly challenging for index structures due to the sparsity of the space and the high intrinsic dimensionality. These datasets allow us to assess system performance across varying dimensionalities, directly testing the scalability of our slab-based memory management against the contiguous memory model of the baseline (Faiss).

4.5.1 High-Throughput Ingestion. As illustrated in Figure 5, SIVF demonstrates superior ingestion scalability. On the SIFT1M dataset,

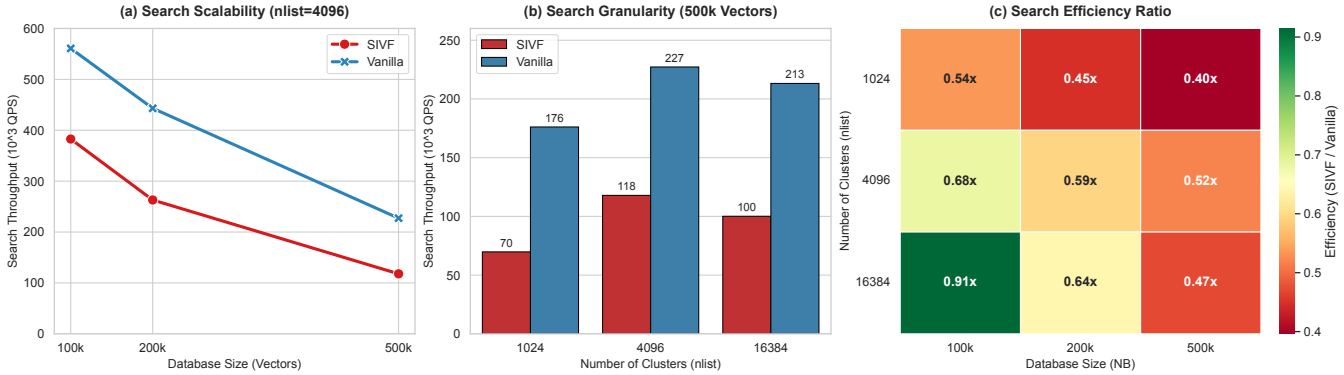

Figure 3: Performance evaluation of vector search.

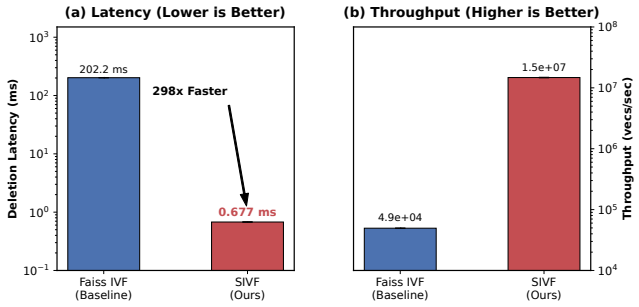

Figure 4: Performance comparison of vector deletion between Faiss IVF (Baseline) and SIVF.

SIVF achieves an ingestion rate of approximately 3.78M vec/s, representing a 105× speedup over the baseline (35.9K vec/s). This advantage persists even with the high-dimensional GIST1M dataset. While the baseline performance drops to 23K vec/s due to the increased cost of memory reallocation and fragmentation management for large vectors, SIVF sustains a throughput of over 850K vec/s. This 36× speedup on GIST1M confirms that our pre-allocated slab architecture effectively mitigates the overhead of dynamic resizing, allowing the system to near-saturate the GPU memory bandwidth even under heavy write loads.

4.5.2 Low-Latency Deletion. The most significant performance divergence is observed in deletion latency, shown in Figure 6. Since the baseline requires a full CPU-GPU roundtrip to reorganize the index, its latency is bound by PCIe bandwidth and index size. Consequently, deleting a batch of 10K vectors takes 1.6 seconds for SIFT1M and a prohibitive 11.8 seconds for GIST1M.

In contrast, SIVF performs deletion in-place via a GPU-native validity bitmap. This decouples deletion latency from both the index size and vector dimensionality. SIVF maintains a sub-millisecond latency ($\approx$ 0.89 ms) across both datasets, achieving a $1,900\times$ improvement on SIFT1M and a staggering $13,300\times$ improvement on GIST1M. This result validates SIVF as a viable solution for real-time streaming scenarios where data freshness is critical.

4.5.3 Search Performance and Trade-offs. Figure 7 presents the query performance (QPS) at comparable recall levels (Recall@10 $\approx$ 90%). On SIFT1M, SIVF achieves a 1.5× higher throughput than the baseline (40.9K vs. 26.7K QPS), benefiting from efficient warp-level parallelism and balanced cluster assignment.

However, on the high-dimensional GIST1M dataset, we observe a performance trade-off. SIVF achieves approximately 37% of the baseline's query throughput (1.3K vs. 3.6K QPS). This reduction is attributed to the non-contiguous memory access pattern inherent in the linked-slab design, which reduces memory coalescing efficiency when fetching large 960-dimensional vectors. We argue that this is an acceptable compromise for streaming systems, where the orders-of-magnitude improvements in mutability (ingestion and deletion) outweigh the moderate cost in raw search throughput.

4.6 Parameter Sensitivity

We evaluate the robustness of SIVF by sweeping three key system parameters that directly affect capacity headroom and update efficiency: (i) the vector capacity factor ($maxvec\_factor$), (ii) the slab pool redundancy ($slab\_factor$), and (iii) the deletion batch size. Intuitively, $maxvec\_factor$ controls the logical headroom of the index (i.e., how aggressively the system provisions capacity relative to the target active set), while $slab\_factor$ determines the amount of pre-allocated slab memory available for absorbing bursty insertions and transient fragmentation. The deletion batch size reflects an application-level knob that trades off amortization efficiency versus the freshness of deletions. Figure 8 and Figure 9 summarize the impact of these parameters on insertion throughput and deletion latency, respectively.

Impact of Slab Management. As shown in Figure 8, SIVF maintains a consistent insertion throughput of approximately 2.7M to 3.2M vec/s across a broad range of configurations. Increasing $slab\_factor$ from 1.0 to 1.3 generally improves performance under high-velocity streams. We attribute this to two effects: (1) a larger slab pool reduces the likelihood of exhausting readily available slabs during bursts, avoiding disruptive on-the-fly allocations or refill paths; and (2) it mitigates contention in slab acquisition by increasing the pool of free slabs, thereby reducing retries in atomic operations on shared allocator metadata. In contrast, SIVF is relatively

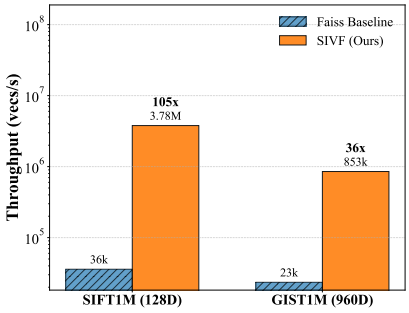

Figure 5: Ingestion throughput.

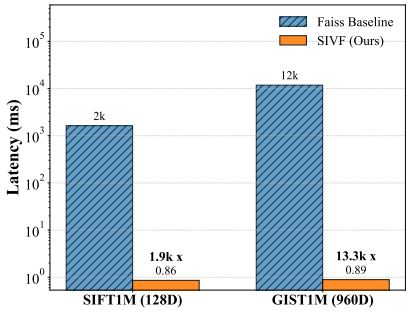

Figure 6: Deletion latency.

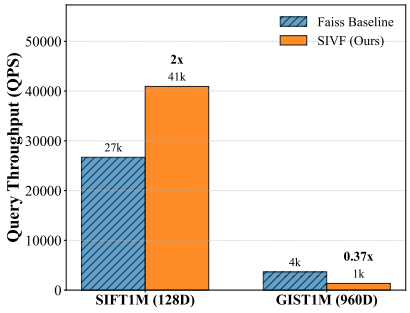

Figure 7: Search performance (QPS).

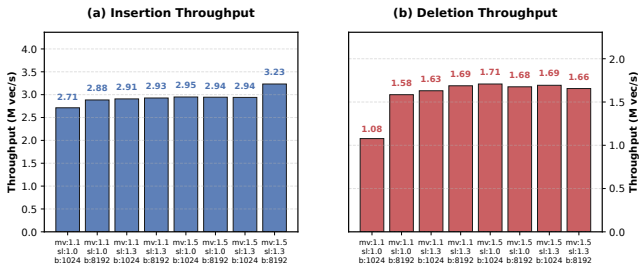

Figure 8: Throughput sensitivity analysis across varying memory factors and batch sizes.

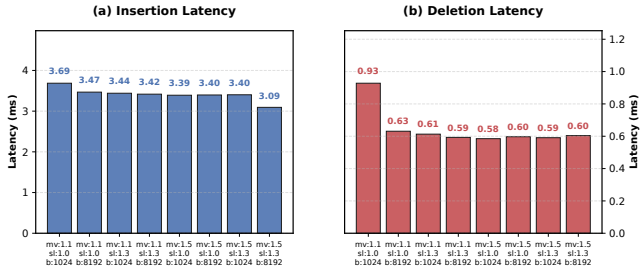

Figure 9: Latency sensitivity analysis demonstrating stable sub-millisecond performance.

insensitive to *maxvec_factor*, indicating that our slab abstraction effectively decouples logical capacity provisioning from immediate GPU memory pressure. Concretely, once slabs are pre-allocated, the dominant insertion costs are bounded by (i) writing the vector payload into a slab slot, (ii) updating lightweight metadata (e.g., validity bits), and (iii) inserting the ID-to-address mapping. These operations remain stable even when the logical capacity headroom changes, provided that the slab pool itself is sufficiently provisioned.

Effect of Deletion Batch Size. Deletion batch size primarily affects amortization rather than per-vector work. For small batches, fixed overheads (kernel launch, lookup setup, and address-table access) become more visible, whereas larger batches amortize these costs and improve effective throughput. This behavior is consistent with SIVF's O(1) logical deletion mechanism: each deletion performs an address-table lookup followed by a single atomic bit update in the slab validity bitmap, without triggering compaction or data movement. In practice, applications can select a batch size that balances deletion freshness and update efficiency: smaller batches minimize staleness (vectors remain searchable for less time after being marked for deletion), while larger batches maximize amortization when strict immediacy is not required.

Stable Sub-Millisecond Deletion. The most critical observation from Figure 9 is that deletion latency remains strictly within the sub-millisecond range (0.58 ms to 0.93 ms) across all tested configurations. This robustness stems from the fact that deletions do not depend on the current occupancy of inverted lists, nor do they require structural reorganization (e.g., compaction, re-clustering, or CPU–GPU roundtrips). Even when processing a deletion batch of 1,000 vectors, the system achieves a peak throughput of over 1.7M vec/s, demonstrating that SIVF can sustain high-rate churn workloads with tight latency constraints.

4.7 End-to-End Streaming Performance of Sliding Windows

While isolated insertion and deletion micro-benchmarks provide insights into atomic operation costs, they do not fully capture the dynamics of a continuous streaming system. To evaluate the practical applicability of SIVF in real-time scenarios, we designed a *Sliding Window* benchmark that mimics a production environment. The system maintains a fixed window of active vectors (denoted as W) in the GPU memory. For every update step, a new batch of vectors (B) is ingested, and the oldest batch (B) is evicted to maintain the window size W. We evaluated two datasets: SIFT1M ($d = 128$, $W = 200$K, $B = 10$K) and GIST1M ($d = 960$, $W = 100$K, $B = 5$K). Figure 10 illustrates the latency per update step for both the Faiss baseline and SIVF.

The results reveal a limitation in the standard static index architecture. For the Faiss baseline, every update step triggers a full reconstruction cycle where the GPU index must be copied to the CPU, modified, and re-uploaded. This roundtrip overhead results in substantial system freezes, averaging 355 ms for SIFT1M and over 1.1 s for GIST1M per update. Such latency renders the baseline unsuitable for latency-sensitive streaming applications, as the indexing pipeline becomes blocked for significant durations.

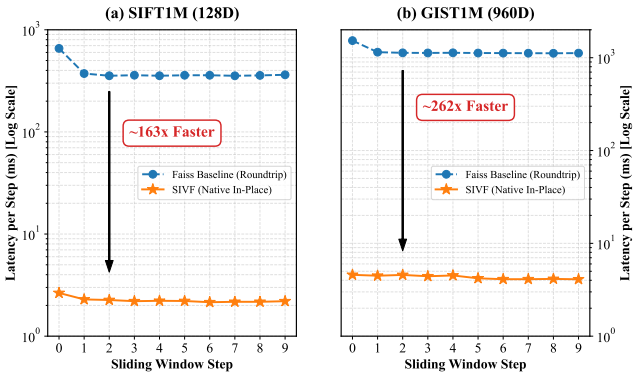

Figure 10: End-to-End Streaming Performance.

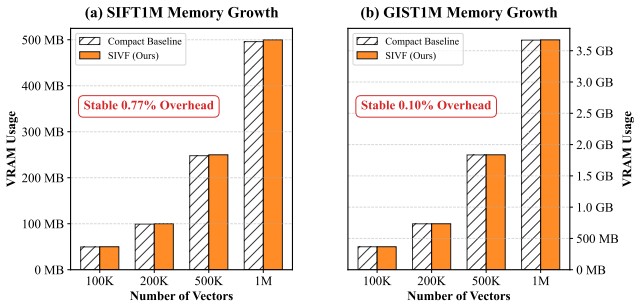

Figure 11: Memory Efficiency and Scalability.

In contrast, SIVF demonstrates superior stability and performance. By leveraging its slab-based memory management and native in-kernel deletion, SIVF executes the entire sliding window update strictly within the GPU. The total latency per step is reduced to approximately 2.2 ms for SIFT1M and 4.2 ms for GIST1M. This represents a speedup of 161× and 266×, respectively.

Notably, the performance gap widens as the data dimensionality increases. In the 960-dimensional GIST1M task, the baseline suffers from PCIe bandwidth saturation due to the massive data movement required for the roundtrip. SIVF avoids this bottleneck entirely, maintaining single-digit millisecond latency even under high-dimensional workloads. These results confirm that SIVF effectively transforms the GPU IVF index from a static search structure into a fully dynamic, real-time streaming engine.

4.8 Memory Efficiency and Scalability

A common concern with dynamic graph-based or list-based indices is the potential for memory bloating due to structural overhead, such as pointers and metadata, as well as fragmentation. To quantify the space complexity of SIVF, we conducted a memory footprint analysis comparing the allocated VRAM usage of SIVF against a theoretically compact array baseline across varying dataset sizes ranging from 100K to 1M vectors.

Figure 11 illustrates the memory growth trends for both SIFT1M and GIST1M. The results demonstrate that SIVF exhibits deterministic linear scalability with negligible structural overhead. For SIFT1M ($d = 128$), the overhead stabilizes at 0.77%, while for the high-dimensional GIST1M ($d = 960$), it is merely 0.10%. This efficiency stems from our coarse-grained slab design, where the metadata cost (128-byte header) is amortized over a batch of 32 vectors. Consequently, SIVF achieves orders-of-magnitude performance gains in dynamic operations without imposing significant storage penalties compared to static, compact indices.

Furthermore, we verified the efficacy of our memory reclamation strategy. In a stress test involving the deletion of 50% of the dataset followed by immediate re-insertion, SIVF maintained a constant memory footprint without triggering OS-level deallocation. The deletion operation completed in sub-millisecond time per batch (e.g., 4.04 ms for 100K GIST vectors), confirming that memory slots are logically reclaimed via bitmap toggling and immediately available for reuse. This "zero-cost" reclamation ensures that SIVF can sustain long-running streaming workloads without suffering from memory leaks or fragmentation-induced bloat.

5 Conclusion

In this work, we addressed the fundamental mismatch between the static design of existing GPU-accelerated ANN indices and the dynamic requirements of real-time streaming applications. Through rigorous benchmarking, we identified the "roundtrip bottleneck", i.e., the necessity of transferring index data across the PCIe bus for modification, as the primary inhibitor of low-latency eviction. To overcome this limitation, we proposed a new index architecture, namely SIVF, that relocates memory management responsibilities entirely onto the GPU. SIVF enables high-throughput and in-place mutability directly in VRAM. By implementing a slab-based memory allocator, a lock-free validity bitmap, and a GPU-resident address translation table, SIVF decouples index maintenance from the host CPU. Our evaluation on the SIFT1M and GIST1M datasets demonstrates that SIVF transforms the landscape of streaming vector search: it accelerates data deletion by up to 13,300× (sub-millisecond latency) and improves ingestion throughput by over 36× compared to the industry-standard Faiss library. These results confirm that moving memory management responsibilities from the CPU to the GPU is not only feasible but essential for the next generation of real-time neural search systems.

While SIVF successfully resolves the mutability bottleneck, several avenues remain for further optimization and expansion. To address the search performance trade-off observed in high-dimensional datasets like GIST1M, we plan to explore adaptive slab sizing and page-aligned super-slabs to restore memory coalescing efficiency for vectors where their dimension is larger than 1K. Furthermore, we aim to extend the architecture to support multi-GPU sharding via NVLink and leverage unified memory techniques, enabling SIVF to scale to extreme-scale datasets that exceed single-card VRAM capacity. Finally, we intend to integrate metadata-filtered search directly into the slab structure, allowing for thread-level pre-filtering and more complex query processing without incurring additional memory lookup overhead.

Acknowledgment

Results presented in this paper were obtained using the Chameleon testbed supported by the National Science Foundation.

References

[1] Chatzakis, M., Papakonstantinou, Y., and Palpanas, T. Darth: Declarative recall through early termination for approximate nearest neighbor search. *Proc. ACM Manag. Data 3*, 4 (Sept. 2025).

[2] Chu, G., He, Y., Dong, L., Ding, Z., Chen, D., Bai, H., Wang, X., and Hu, C. Efficient algorithm design of optimizing spmv on gpu. In *Proceedings of the 32nd International Symposium on High-Performance Parallel and Distributed Computing* (New York, NY, USA, 2023), HPDC '23, Association for Computing Machinery, p. 115–128.

[3] Chung, J. W., Lin, H., and Zhao, W. Locality-sensitive indexing for graph-based approximate nearest neighbor search. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval* (New York, NY, USA, 2025), SIGIR '25, Association for Computing Machinery, p. 2418–2428.

[4] Deng, L., Chen, P., Zeng, X., Wang, T., Zhao, Y., and Zheng, K. Efficient data-aware distance comparison operations for high-dimensional approximate nearest neighbor search. *Proc. VLDB Endow. 18*, 3 (Nov. 2024), 812–821.

[5] Douze, M., Guzhva, A., Deng, C., Johnson, J., Szilvasy, G., Mazaré, P.-E., Lomeli, M., Hosseini, L., and Jégou, H. The faiss library.

[6] Engels, J., Landrum, B., Yu, S., Dhulipala, L., and Shun, J. Approximate nearest neighbor search with window filters. In *Forty-first International Conference on Machine Learning* (2024).

[7] Gao, J., Gou, Y., Xu, Y., Yang, Y., Long, C., and Wong, R. C.-W. Practical and asymptotically optimal quantization of high-dimensional vectors in euclidean space for approximate nearest neighbor search. *Proc. ACM Manag. Data 3*, 3 (June 2025).

[8] Gao, J., and Long, C. Rabitq: Quantizing high-dimensional vectors with a theoretical error bound for approximate nearest neighbor search. *Proc. ACM Manag. Data 2*, 3 (May 2024).

[9] Gong, Z., Zeng, Y., and Chen, L. Accelerating approximate nearest neighbor search in hierarchical graphs: Efficient level navigation with shortcuts. *Proc. VLDB Endow. 18*, 10 (June 2025), 3518–3530.

[10] Gou, Y., Gao, J., Xu, Y., and Long, C. Symphonyqg: Towards symphonious integration of quantization and graph for approximate nearest neighbor search. *Proc. ACM Manag. Data 3*, 1 (Feb. 2025).

[11] Hua, Z., Mo, Q., Yao, Z., Cui, L., Liu, X., Wang, G., Wei, Z., Liu, X., Tang, T., Liu, S., and Qu, L. Dynamically detect and fix hardness for efficient approximate nearest neighbor search. *Proc. ACM Manag. Data 3*, 6 (Dec. 2025).

[12] Hui, X., Xu, Y., Guo, Z., and Shen, X. Esg: Pipeline-conscious efficient scheduling of dnn workflows on serverless platforms with shareable gpus. In *Proceedings of the 33rd International Symposium on High-Performance Parallel and Distributed Computing* (New York, NY, USA, 2024), HPDC '24, Association for Computing Machinery, p. 42–55.

[13] Hui, X., Xu, Y., and Shen, X. Fluidfaas: A dynamic pipelined solution for serverless computing with strong isolation-based gpu sharing. In *Proceedings of the 34th International Symposium on High-Performance Parallel and Distributed Computing* (New York, NY, USA, 2025), HPDC '25, Association for Computing Machinery.

[14] Hwang, S., Lee, E., Oh, H., and Yi, Y. Fasop: Fast yet accurate automated search for optimal parallelization of transformers on heterogeneous gpu clusters. In *Proceedings of the 33rd International Symposium on High-Performance Parallel and Distributed Computing* (New York, NY, USA, 2024), HPDC '24, Association for Computing Machinery, p. 253–266.

[15] Indyk, P., and Xu, H. Worst-case performance of popular approximate nearest neighbor search implementations: Guarantees and limitations. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023* (2023), A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, Eds.

[16] Jääsaari, E., Hyvönen, V., and Roos, T. Lorann: Low-rank matrix factorization for approximate nearest neighbor search. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024* (2024), A. Globersons, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. M. Tomczak, and C. Zhang, Eds.

[17] Jiang, M., Yang, Z., Zhang, F., Hou, G., Shi, J., Zhou, W., Li, F., and Wang, S. Digra: A dynamic graph indexing for approximate nearest neighbor search with range filter. *Proc. ACM Manag. Data 3*, 3 (June 2025).

[18] Johnson, J., Douze, M., and Jégou, H. Billion-scale similarity search with gpus. *IEEE Transactions on Big Data 7*, 3 (2021), 535–547.

[19] Jung, S., Park, Y., Lee, H., Oh, Y. H., and Lee, J. W. Angular distance-guided neighbor selection for graph-based approximate nearest neighbor search. In *Proceedings of the ACM on Web Conference 2025* (New York, NY, USA, 2025), WWW '25, Association for Computing Machinery, p. 4014–4023.

[20] Jégou, H., Douze, M., and Schmid, C. Product quantization for nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence 33*, 1 (2011), 117–128.

[21] Keahey, K., Anderson, J., Zhen, Z., Riteau, P., Ruth, P., Stanzione, D., Cevik, M., Colleran, J., Gunawi, H. S., Hammock, C., Mambretti, J., Barnes, A., Halbach, F., Rocha, A., and Stubbs, J. Lessons learned from the chameleon testbed. In *Proceedings of the 2020 USENIX Annual Technical Conference (USENIX ATC '20)*. USENIX Association, July 2020.

[22] Laut, S., Borrell, R., and Casas, M. Extending sparse patterns to improve inverse preconditioning on gpu architectures. In *Proceedings of the 33rd International Symposium on High-Performance Parallel and Distributed Computing* (New York, NY, USA, 2024), HPDC '24, Association for Computing Machinery, p. 200–213.

[23] Leyba, K., Hofmeyr, S., Forrest, S., Cannon, J., and Moses, M. Simcov-gpu: Accelerating an agent-based model for exascale. In *Proceedings of the 33rd International Symposium on High-Performance Parallel and Distributed Computing* (New York, NY, USA, 2024), HPDC '24, Association for Computing Machinery, p. 322–333.

[24] Li, M., Yan, X., Lu, B., Zhang, Y., Cheng, J., and Ma, C. Attribute filtering in approximate nearest neighbor search: An in-depth experimental study. *Proc. ACM Manag. Data 3*, 6 (Dec. 2025).

[25] Li, X., Laguna, I., Fang, B., Swirydowicz, K., Li, A., and Gopalakrishnan, G. Design and evaluation of gpu-fpx: A low-overhead tool for floating-point exception detection in nvidia gpus. In *Proceedings of the 32nd International Symposium on High-Performance Parallel and Distributed Computing* (New York, NY, USA, 2023), HPDC '23, Association for Computing Machinery, p. 59–71.

[26] Li, Z., Ke, X., Zhu, Y., Yu, B., Zheng, B., and Gao, Y. Scalable graph indexing using gpus for approximate nearest neighbor search. *Proc. ACM Manag. Data 3*, 6 (Dec. 2025).

[27] Liang, A., Zhang, P., Yao, B., Chen, Z., Song, Y., and Cheng, G. Unify: Unified index for range filtered approximate nearest neighbors search. *Proc. VLDB Endow. 18*, 4 (Dec. 2024), 1118–1130.

[28] Liu, M., Zhong, S., Yang, Q., Han, Y., Liu, X., and Ma, Y. Webanns: Fast and efficient approximate nearest neighbor search in web browsers. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval* (New York, NY, USA, 2025), SIGIR '25, Association for Computing Machinery, p. 2483–2492.

[29] Lu, K., Xiao, C., and Ishikawa, Y. Probabilistic routing for graph-based approximate nearest neighbor search. In *Forty-first International Conference on Machine Learning* (2024).

[30] Maurya, A., Rafique, M. M., Tonellot, T., AlSalem, H. J., Cappello, F., and Nicolae, B. Gpu-enabled asynchronous multi-level checkpoint caching and prefetching. In *Proceedings of the 32nd International Symposium on High-Performance Parallel and Distributed Computing* (New York, NY, USA, 2023), HPDC '23, Association for Computing Machinery, p. 73–85.

[31] Miao, D., Laguna, I., and Rubio-González, C. Floatguard: Efficient whole-program detection of floating-point exceptions in amd gpus. In *Proceedings of the 34th International Symposium on High-Performance Parallel and Distributed Computing* (New York, NY, USA, 2025), HPDC '25, Association for Computing Machinery.

[32] Peng, Z., Qiao, M., Zhou, W., Li, F., and Deng, D. Dynamic range-filtering approximate nearest neighbor search. *Proc. VLDB Endow. 18*, 10 (June 2025), 3256–3268.

[33] Peng, Z., Thomadakis, P., Pienaar, J., and Kestor, G. Liteform: Lightweight and automatic format composition for sparse matrix-matrix multiplication on gpus. In *Proceedings of the 34th International Symposium on High-Performance Parallel and Distributed Computing* (New York, NY, USA, 2025), HPDC '25, Association for Computing Machinery.

[34] Qiu, R., and Tang, J. Efficient approximate nearest neighbor search via hemisphere centroids graph. *Proc. ACM Manag. Data 3*, 6 (Dec. 2025).

[35] Sun, P., Simcha, D., Dopson, D., Guo, R., and Kumar, S. SOAR: improved indexing for approximate nearest neighbor search. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023* (2023), A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, Eds.

[36] Tran, A., Laguna, I., and Gopalakrishnan, G. Fpboxer: Efficient input-generation for targeting floating-point exceptions in gpu programs. In *Proceedings of the 33rd International Symposium on High-Performance Parallel and Distributed Computing* (New York, NY, USA, 2024), HPDC '24, Association for Computing Machinery, p. 83–93.

[37] Voruganti, S., and Özsu, M. T. Mirage-anns: Mixed approach graph-based indexing for approximate nearest neighbor search. *Proc. ACM Manag. Data 3*, 3 (June 2025).

[38] Wang, M., Lv, L., Xu, X., Wang, Y., Yue, Q., and Ni, J. An efficient and robust framework for approximate nearest neighbor search with attribute constraint. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023* (2023), A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, Eds.

[39] Wang, Z., Zhang, J., and Hu, W. Wow: A window-to-window incremental index for range-filtering approximate nearest neighbor search. *Proc. ACM Manag. Data*

3, 6 (Dec. 2025).

[40] Wei, J., Lee, X., Liao, Z., Palpanas, T., and Peng, B. Subspace collision: An efficient and accurate framework for high-dimensional approximate nearest neighbor search. *Proc. ACM Manag. Data 3*, 1 (Feb. 2025).

[41] Xu, Q., Zhang, F., Li, C., Cao, L., Chen, Z., Zhai, J., and Du, X. Harmony: A scalable distributed vector database for high-throughput approximate nearest neighbor search. *Proc. ACM Manag. Data 3*, 4 (Sept. 2025).

[42] Yang, M., Cai, Y., and Zheng, W. CSPG: crossing sparse proximity graphs for approximate nearest neighbor search. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024* (2024), A. Globersons, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. M. Tomczak, and C. Zhang, Eds.

[43] Zhang, B., Tian, J., Di, S., Yu, X., Feng, Y., Liang, X., Tao, D., and Cappello, F. Fz-gpu: A fast and high-ratio lossy compressor for scientific computing applications on gpus. In *Proceedings of the 32nd International Symposium on High-Performance Parallel and Distributed Computing* (New York, NY, USA, 2023), HPDC '23, Association for Computing Machinery, p. 129–142.

[44] Zhang, F., Jiang, M., Hou, G., Shi, J., Fan, H., Zhou, W., Li, F., and Wang, S. Efficient dynamic indexing for range filtered approximate nearest neighbor search. *Proc. ACM Manag. Data 3*, 3 (June 2025).

[45] Zhong, X., Li, H., Jin, J., Yang, M., Chu, D., Wang, X., Shen, Z., Jia, W., Gu, G., Xie, Y., Lin, X., Shen, H. T., Song, J., and Cheng, P. Vsag: An optimized search framework for graph-based approximate nearest neighbor search. *Proc. VLDB Endow. 18*, 12 (Aug. 2025), 5017–5030.

A Appendix

A.1 Codebase Implementation and Modifications

Our implementation is built upon the Faiss library. To integrate the SIVF architecture, we introduced significant modifications to the GPU module and developed a comprehensive benchmarking suite. Table 1 summarizes the key source files added or modified in the repository based on our development history.

The SIVF implementation entails a non-trivial engineering effort. Based on Git history statistics, the project introduces approximately 9,000 lines of code changes (insertions and deletions) spanning over 45 source files. Specifically, more than 3,000 lines are devoted to the core GPU index logic, including new CUDA kernels for lock-free insertion, bitmap-based deletion, and collaborative search, alongside a custom slab-based memory allocator (`SlabManager`). A substantial portion of the codebase (over 5,000 lines) is dedicated to the experimental ecosystem. This includes granular micro-benchmarks for specific operations (addition, deletion, search), complex end-to-end scenarios (sliding window streaming, memory fragmentation stress tests), and hyperparameter sensitivity analysis. The remaining code comprises build configurations (CMake), dataset loaders for SIFT1M and GIST1M, and Python scripts for automated result visualization, ensuring reproducibility and ease of integration.

A.2 Lessons Learned

Developing a fully dynamic, GPU-resident index presented several unique engineering challenges. We summarize the key lessons learned during the development of SIVF:

- *Implicit Synchronization Risks.* In the early stages of development, we encountered non-deterministic data corruption during high-concurrency ingestion. We traced this to the GPU's relaxed memory consistency model. Writes to a new slab were sometimes not visible to other streaming multiprocessors before the slab was linked to the list. We learned that explicit `__threadfence()` instructions are non-negotiable when implementing lock-free data structures on GPUs, even within the same kernel launch.
- *Register Pressure vs. Occupancy.* Our initial search kernel utilized complex template meta-programming to handle various metric types, which bloated the register usage per thread and limited occupancy. We found that manually unrolling loops and using simpler, distinct kernels for Inner Product and L2 distance significantly reduced register pressure, allowing more warps to be active simultaneously and hiding global memory latency more effectively.
- *The Cost of Allocators.* We initially attempted to use the built-in CUDA `malloc` for dynamic node creation. This proved disastrous for performance due to internal locking within the driver. This failure motivated the design of the custom `SlabManager`. The lesson is that for high-throughput system code, general-purpose memory allocators on the device are rarely sufficient; domain-specific, arena-based allocators are essential for performance.

Table 1: Summary of Source Files and Modifications in SIVF Project

File Path	*Description*
Core SIVF Architecture & Memory Management	
`faiss/gpu/GpuIndexSIVF.h`	Header definition for the SIVF index class, inheriting from `GpuIndex`.
`faiss/gpu/GpuIndexSIVF.cu`	Manages host-device synchronization and overrides standard operations.
`faiss/gpu/impl/SlabManager.cuh`	Defines the device-side slab structures, arena pointers, and metadata layout.
`faiss/gpu/impl/SlabManager.cu`	Slab-based Dynamic Memory Allocator (SDMA) and memory pool management.
`faiss/gpu/impl/SIVFStructs.cuh`	Shared data structures and metadata definitions used across host and device code.
CUDA Kernels (Ingestion, Search, Deletion)	
`faiss/gpu/impl/SIVFAppend.cuh`	Device function templates for the ingestion logic.
`faiss/gpu/impl/SIVFAppend.cu`	Lock-free insertion kernel, atomic slot reservation, and slab expansion logic.
`faiss/gpu/impl/SIVFSearch.cuh`	Warp-Level Collaborative Search (WLCS) and shared memory reduction.
`faiss/gpu/impl/SIVFSearch.cu`	Search kernel, including distance computation and heap aggregation.
`faiss/gpu/SIVFDeletion.cu`	Bitmap-based lazy eviction and deletion kernels.
Experimental Benchmarks (C++)	
`hpdic/experiment/test_sivf_sift_add.cpp`	Throughput benchmark for SIFT1M vector ingestion.
`hpdic/experiment/test_sivf_gist_add.cpp`	Throughput benchmark for GIST1M vector ingestion.
`hpdic/experiment/test_sivf_sift_search.cpp`	Latency and recall benchmark for SIFT1M queries.
`hpdic/experiment/test_sivf_gist_search.cpp`	Latency and recall benchmark for GIST1M queries.
`hpdic/experiment/test_sivf_sift_delete.cpp`	Deletion throughput and stability test for SIFT1M.
`hpdic/experiment/test_sivf_gist_delete.cpp`	Deletion throughput and stability test for GIST1M.
`hpdic/experiment/test_sivf_sliding.cpp`	End-to-End Streaming Benchmark simulating a sliding window workload.
`hpdic/experiment/test_sivf_memory.cpp`	Memory Scalability Benchmark quantifying VRAM overhead and reuse.
`hpdic/experiment/test_sivf_sensitivity.cpp`	Parameter Study evaluating performance sensitivity to batch/slab settings.
`hpdic/experiment/benchmark_baseline.cpp`	Comparison baseline implementing CPU-GPU roundtrip logic for standard IVF.
Data Utilities & Visualization	
`hpdic/experiment/sift/sift_loader.h`	Optimized raw vector loader for SIFT1M dataset.
`hpdic/experiment/gist/gist_loader.h`	Optimized raw vector loader for GIST1M dataset.
`hpdic/experiment/plot_add.py`	Python script for generating ingestion throughput figures.
`hpdic/experiment/plot_search.py`	Python script for generating QPS vs. Recall trade-off figures.
`hpdic/experiment/plot_delete.py`	Python script for generating deletion latency figures.
`hpdic/experiment/plot_sliding.py`	Python script for generating sliding window streaming performance figures.
`hpdic/experiment/plot_memory.py`	Python script for generating memory scalability and overhead figures.
`hpdic/experiment/plot_sensitivity.py`	Python script for generating parameter sensitivity heatmaps or curves.